JetBot Workshop — Mentor Troubleshooting Cheat Sheet

Quick-reference for group mentors. Try the fix in order; if it doesn't resolve in ~2 minutes, flag Curtis/lead tech rather than digging deeper — keep the group moving on something else in the meantime.

Can't connect / Jupyter won't load

Symptom: Browser can't reach the JetBot's Jupyter URL at all.

- Confirm your laptop is on the AI Lab WiFi network (hidden SSID — must be entered manually, easy to fumble). Reconnect if needed.
- Confirm you're using the correct hostname/IP for *your* JetBot (check the OLED display on the robot — it shows its IP).
- Try a hard refresh / new browser tab before restarting anything.

Symptom: Jupyter loads but gamepad doesn't respond in the notebook.

- Gamepad API requires a secure context — this only works over `localhost`. If it's not pre-configured, this needs an SSH port-forward tunnel set up in advance; flag the lead tech rather than troubleshooting live.

Camera errors

Symptom: `RuntimeError: Could not read image from camera` (or camera feed frozen/black in notebook).

- Restart the notebook kernel first (Kernel → Restart) — resolves it a good chunk of the time.
- If there's no image or the feed is all dark, check whether the lens cap is still on the camera and remove it.
- Check for a loose camera cable: look at both ends (camera module and Jetson board) and confirm the cable is fully seated and hasn't worked its way out. If it's visibly loose, power the JetBot off before reseating it.
- If that doesn't work, the camera needs to be reset on the device. Try the fixes below in order; after each one, restart the kernel and rerun the notebook from the beginning. (There are several different underlying causes for this same error, so don't assume last time's fix applies.)
  1. Run the camera reset notebook in the `/utilities` folder: `jetbot-camera-reset.ipynb`
  2. Run the camera reset bash script. SSH into the JetBot from a command window, go to the home folder, and run the script:

     ```
     ssh nvidia@<ip address>
     cd ~
     bash jetbot-camera-reset.sh
     ```

  3. Last resort: power-cycle the JetBot. In the SSH session, run `sudo poweroff`. Once the JetBot has shut down (it's off, but the power switch is still on), flip the switch off, then back on.
- Don't unplug/replug the camera cable as a first move — it's rarely the cause and risks a bad reseat.

Robot won't move / motors unresponsive

- Check the JetBot is powered on and battery isn't critically low (OLED display shows battery status).
- Rerun the motor setup/init cell in the notebook — sometimes the motor object needs to be re-instantiated after a kernel restart.
- Confirm the correct cell order was run — motion commands run out of order (e.g., before the robot object is created) will silently no-op or error.

Gamepad not responding

- Make sure the correct gamepad index number is specified in the notebook cell.
- If the index is correct and the gamepad still isn't responding, turn the gamepad switch off, then back on.
- If the gamepad never responds at all, see the localhost/secure-context note under "Can't connect / Jupyter won't load" above.

OLED display frozen, blank, or not showing IP

Symptom: OLED display is frozen or blank.

- This is usually an I2C bus conflict, not a hardware fault. Don't power-cycle as a first step — flag the lead tech, it's a known service-level fix.
- The IP display service code has been updated to be more robust, so this should now happen much less often.

Symptom: OLED shows the hostname but no IP address.

- Replace the JetBot with the spare so the group can keep working, then debug the problem unit offline.
- Connect the JetBot to a USB keyboard, mouse, and HDMI monitor and log in to a terminal session.
- Check whether it's connected to the hidden network. In bash: `iwconfig wlan0` and check the ESSID.
- If it's not connected to the hidden network, connect it.
- If it is connected, get the IP address. In bash: `ip address` and look for the `inet` address under `wlan0`.
- Only as a last resort, reboot with `sudo reboot`.

Road Following: robot spins in place or won't follow the line

- Check the steering slider/gain value — very small adjustments matter a lot here; encourage students to nudge in small increments, not big jumps.
- If the robot consistently turns the *wrong* way relative to the line, the steering sign may need flipping — flag the lead tech rather than having students guess-and-check indefinitely.
- More/better-distributed training data usually helps more than parameter tweaking. If the model seems confused, collect a few more images at the specific spot it fails, especially curves and edge cases.
- **Run the JetBot under the same conditions the training images were captured in.** The model is very sensitive to lighting, floor reflections, etc. Keep the room the same as during data collection: window shades drawn (or open) the same way, lights on, surrounding furniture and objects in the same place. If the robot suddenly behaves worse than it did earlier, check whether the room changed before touching any settings.
- **Check the camera focus.** Look at the live camera image in the notebook; if it looks blurry, adjust the focus ring on the camera lens a little at a time until the image is sharp (ideally before collecting training images).

Collision Avoidance: robot ignores obstacles, or behaves inconsistently

- Behavior can look weirdly "pose-dependent" — the same obstacle in a slightly different position/angle gets a different result. This is a data quality issue, not a bug: encourage collecting "blocked" images from varied distances/angles, not just straight-on.
- If the model seems to have barely learned anything (near-random behavior), check that both "blocked" and "free" classes have a reasonably balanced number of images.
- **Run the JetBot under the same conditions the training images were captured in.** The model is very sensitive to lighting, floor reflections, etc. Keep the room the same as during data collection: window shades drawn (or open) the same way, lights on, surrounding furniture and objects in the same place. If the robot suddenly behaves worse than it did earlier, check whether the room changed before retraining or changing anything else.
- **Check the camera focus.** Look at the live camera image in the notebook; if it looks blurry, adjust the focus ring on the camera lens a little at a time until the image is sharp (ideally before collecting training images).

Model training is slow / notebook seems stuck

- Training does take real time — don't assume it's frozen. Check for a progress indicator/epoch counter in the output before restarting anything.
- If genuinely stuck (no output changing for several minutes), restart the kernel and rerun from the training cell — no need to redo data collection.
- If training keeps stalling, the kernel dies ("Kernel restarting" / "Dead kernel"), or the whole JetBot becomes sluggish during training, it is probably running out of memory. Creating a swap file on the JetBot usually fixes this. Only needs to be done once per robot — it persists across reboots.
    1. ssh into the jetbot, from a command window: `ssh nvidia@<ip address>` (IP is shown on the OLED display)
    2. check whether swap already exists, and that `/data/projects` is on the microSD card with enough free space:

       ```
       swapon --show
       free -h
       df -h /data/projects
       ```

       The `Filesystem` column from `df` should show `/dev/mmcblk1...` (the microSD card). If it doesn't, stop and flag the lead tech. If a swap file of ~4G is already listed (e.g., `/data/projects/swapfile`), stop here — it's already set up; go back to the restart-kernel step above.
    3. create and enable a 4GB swap file on the microSD card:

       ```
       sudo fallocate -l 4G /data/projects/swapfile
       sudo chmod 600 /data/projects/swapfile
       sudo mkswap /data/projects/swapfile
       sudo swapon /data/projects/swapfile
       ```

    4. make it permanent (only run once — don't add a duplicate line to fstab):

       ```
       grep -q '/data/projects/swapfile' /etc/fstab || echo '/data/projects/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
       ```

    5. verify: `free -h` should now show ~4G under Swap. Then restart the notebook kernel (Kernel → Restart) and rerun from the training cell.
    - If `fallocate` fails or `df -h /data/projects` shows less than ~5G free, don't force it — flag the lead tech.

Escalate immediately (don't try to fix live) if:

- SSH/Jupyter access is broken and reconnecting doesn't fix it
- The robot doesn't power on at all, or shows physical damage
- A fix requires console access to the device, or SSH steps that aren't listed in this sheet
- You're not confident the fix won't eat into another group's time

Have a spare JetBot on hand — if a unit is down for more than a few minutes, swap the group onto the spare rather than troubleshooting hardware live.